arXiv:submit/7412053 [cs.AI] 27 Mar 2026

Improving Parameter Utilization by Sharing Neural Experts Across Transformer Layers

Dian Jiao[*], **Jiaxin Duan**[*], **Shuai Zhao, Jian Wang, Jiabing Leng, Yiran Zhang, Feng Huang**[†]
China Electronics Cloud Technology Co., Ltd.
{jiaodian, duanjiaxin, zhaoshuai}@cestc.cn

Abstract

While Transformer-based architectures have revolutionized Large Language Models (LLMs), existing designs suffer from substantial inter-layer parameter redundancy, where dimensional transformations in Feed-Forward Network (FFN) layers or sparse experts are independently re-learned across network depths. In this paper, we investigate this parameter redundancy problem and propose CS-MoE, a novel architecture that implements cross-layer expert sharing in MoE Transformers. Unlike traditional MoE approaches that rely on layer-isolated experts, CS-MoE equips each Transformer layer with a small set of independent experts while providing simultaneous access to a large, centralized shared expert pool. This hybrid structure enables precise control over both token-level parameter activation and computational budgets (FLOPs) by scaling the shared pool and tuning the activation density. Our architectural innovation allows CS-MoE to transcend traditional parameter capacity limits through dynamic compute allocation. Extensive experiments demonstrate three key advantages: (1) *Superior Efficiency*: Compared to dense baselines, CS-MoE achieves significantly lower perplexity while activating only 55% of the parameters. (2) *Scalable Performance*: With a fixed parameter scale, increasing expert activation counts yields continuous performance gains, effectively utilizing additional FLOPs. (3) *Architectural Convergence*: Under fixed FLOPs, expanding the shared expert pool reduces expert repetition and drives performance toward the upper bound of standard MoEs. Ultimately, CS-MoE establishes a flexible Pareto frontier between computational expenditure and model capacity, offering a high-efficiency alternative to standard MoEs for FLOP-constrained environments.

1 Introduction

The proliferation of Transformer-based architectures has served as a pivotal advancement in artificial intelligence, providing the representational capacity and parallelization necessary to catalyze the success of Large Language Models (LLMs). However, as these models scale toward trillion-parameter regimes, the traditional **Dense Transformer** encounters diminishing marginal returns in efficiency [4, 13]. Recent empirical inquiries into weight pruning and model compression [17, 11] suggest that dense models suffer from significant **activation redundancy**, wherein a substantial portion of parameters contributes negligibly to the final output for any given input [3]. This phenomenon indicates that standard Feed-Forward Networks (FFNs) contain latent capacity that remains underutilized during inference, leading to a suboptimal ratio of information density to computational expenditure.

[*]Equal contribution.
[†]Corresponding author.

Preprint.

To mitigate this inefficiency, the **Mixture-of-Experts (MoE)** paradigm [22, 8] has emerged as the *de facto* standard for scaling LLMs. By substituting monolithic FFN layers with a set of sparse experts and selectively activating a subset (Top-K) per token, MoE architectures effectively decouple total model capacity from active computational cost (FLOPs). Nevertheless, while MoE alleviates the computational burden, its conventional implementation - defined by **layer-isolated experts** - introduces a secondary inefficiency termed **parameter redundancy**. In standard MoE designs, experts are confined to specific layers, preventing the network from exploiting functional recurrence across different stages of the hidden state transformation.

Our research is motivated by a growing body of work that endeavors to dissect MoE architectures [9, 19, 20], which suggests that the rigid isolation of parameters in sparse architectures is often unnecessary. These studies reveal that strategic parameter reuse and dynamic allocation can significantly enhance architectural efficiency without incurring a substantial sacrifice in modeling performance. Inspired by these findings, *we hypothesize that the physical barriers between layers in traditional MoEs lead to an underutilization of expert capacity*, as similar functional transformations are often redundantly learned at multiple depths. In Figure 1, we investigate the potential for cross-layer expert sharing. Empirical validation through zero-shot substitution demonstrates that replacing deep-layer experts with functionally equivalent shallow counterparts - identified via Centered Kernel Alignment (CKA) (see Appendix A for details) - incurs moderate perplexity degradation, significantly outperforming random replacement and confirming that experts are largely interchangeable across depths.

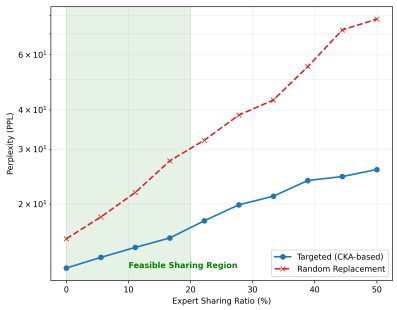


Figure 1: PPL of MoE models with random expert replacement and deliberate expert replacement based on CKA analysis.

Building upon these insights, we propose **CS-MoE**, a novel Transformer architecture that implements a "super-sharing" mechanism to bridge the gap between sparse capacity and parameter efficiency. CS-MoE restructures the traditional expert distribution into a **dual-tier hierarchy** that distinguishes between localized specialization and global functional reuse. Instead of the conventional layer-isolated approach, we introduce a **dual-path execution strategy**: a **Fixed Path** comprising a small set of **Independent Experts** that are unconditionally activated at each layer to capture depth-specific features (e.g., hierarchical syntax or task-specific projections), and a **Dynamic Path** that provides all layers with concurrent access to a centralized **Shared Expert Pool**. By employing a layer-specific router to select proportional experts from this global repository, the architecture allows tokens to invoke universal semantic operators regardless of their position in the network stack. This design principle ensures that the model maintains the hierarchical integrity of the Transformer while simultaneously benefiting from the massive parameter capacity afforded by a shared global pool.

We validate the efficacy of CS-MoE through a series of rigorous experiments ranging from models with 0.6B to 12B parameter scales. Our results demonstrate three primary technical advantages. First, we establish **Superior Parameter Density**; at a fixed computational and parameter budget, CS-MoE achieves a significant reduction in perplexity compared to dense baselines, requiring only **55%** of the parameters to be active per forward pass. Second, we demonstrate **Scalable Compute Performance**, where model performance scales monotonically with the expert activation count (K), allowing CS-MoE to transcend the capacity limitations inherent in traditional fixed-parameter designs. Finally, we characterize the **Flexible Pareto Frontier** of the architecture, showing that in FLOP-constrained environments, CS-MoE provides a high-efficiency alternative that converges toward the performance upper bound of standard MoEs as the shared pool is scaled. Through these contributions, CS-MoE establishes a new equilibrium between computational expenditure and model capacity, providing a scalable pathway for deploying memory-efficient LLMs.

2 Methodology

The architecture of **CS-MoE** is centered on the hypothesis that optimal model capacity is achieved not through independent scaling of layers, but through a hierarchical organization of parameters. By

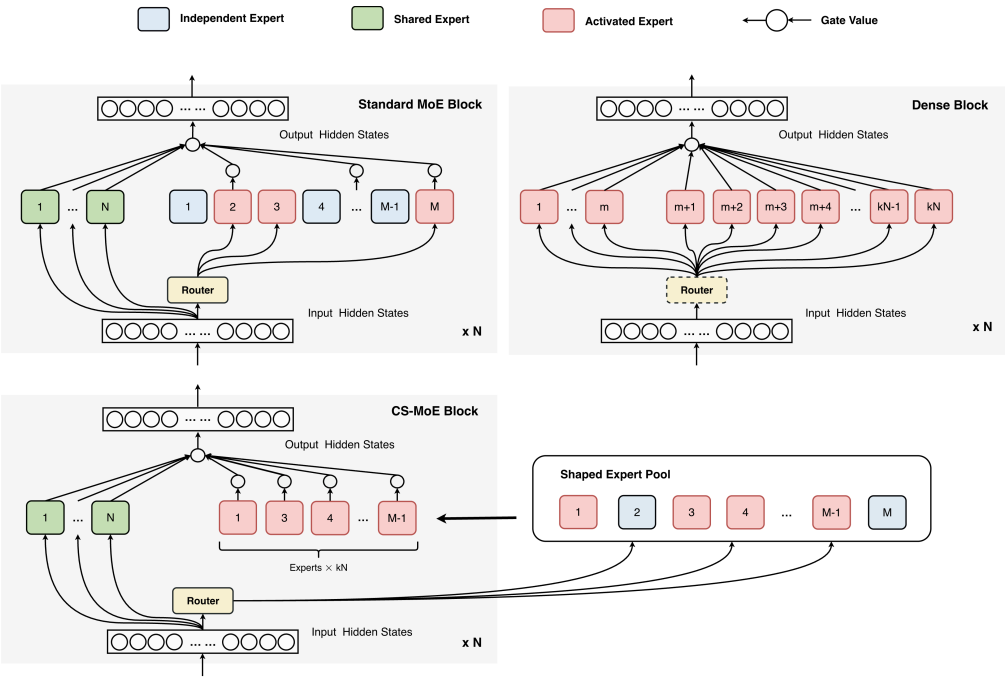


Figure 2: Comparison of Transformer architectures: standard MoE (upper left), dense (upper right), and CS-MoE (lower). The dense Transformer has a virtual router marked in a dashed box.

bifurcating the Feed-Forward Network (FFN) into a dense layer-specific path and a sparse global path, CS-MoE addresses inter-layer redundancy while maintaining the specialized capacity required for deep hierarchical processing.

2.1 Transformers Reviewing

To provide a rigorous foundation for CS-MoE, we formalize the Transformer Feed-Forward Network (FFN) through a generalized Mixture-of-Experts (MoE) framework. A standard **Dense Transformer** represents a constrained instance of this topology where the model employs a single, "always-on" expert ($N = 1, K = 1$) at every layer. In contrast, a **Sparse MoE Transformer** scales the parameter count by utilizing a larger pool of experts per layer ($N > 1$) but restricts computation by activating only a small subset ($K \ll N$).

CS-MoE generalizes these paradigms into a hybrid, dual-tier topology. It posits that a fixed portion of the transformation parameters should be "anchored" to specific depths to capture localized features, while a significantly larger, centralized pool should be shared across all layers to handle recurrent semantic patterns. By breaking the physical isolation between layers, CS-MoE enables a single expert to participate in the computation of multiple stages of the network, thereby maximizing parameter utility and facilitating a more efficient equilibrium between model capacity and computational expenditure.

2.2 CS-MoE Architecture

As illustrated in Figure 2, CS-MoE restructures each Transformer layer $l \in \{1, \ldots, L\}$ into a dual-path execution model: a **Fixed Path** consisting of layer-specific Independent Experts and a **Dynamic Path** drawing from a global Shared Expert Pool. The total expert set available to the network is partitioned as follows:

$$\mathcal{E}_{total} = \left(\bigcup_{l=1}^{L} \mathcal{E}_{indep}^{(l)} \right) \cup \mathcal{E}_{shared}$$

where $\mathcal{E}_{indep}^{(l)} = \{e_1^{(l)}, \ldots, e_N^{(l)}\}$ contains N independent experts private to layer l, and $\mathcal{E}_{shared} = \{s_1, \ldots, s_M\}$ is a large, centralized repository of M experts accessible by every layer in the Transformer stack. In this configuration, the execution logic for each layer follows two distinct principles:

- **Routing-free Fixed Path:** Each layer l activates its own N **Independent Experts** ($\mathcal{E}_{indep}^{(l)}$) unconditionally for every token. This ensures a baseline of depth-specific transformation capacity without the overhead of routing, effectively maintaining the reliability of dense architectures.
- **Sparse Dynamic Path:** The routing mechanism in CS-MoE is exclusively restricted to the **Shared Expert Pool**. For each token at layer l, a layer-specific router selects and activates kN experts from $\mathcal{E}_{shared}$. Crucially, because the shared pool is universal, a specific shared expert s_j may be activated simultaneously across multiple layers for the same or different tokens, allowing for extreme longitudinal parameter reuse.

Consequently, the total computation of a single layer is composed of N independent experts and kN routed shared experts, resulting in a total of $(k+1)N$ activated experts per layer. This hybrid structure allows CS-MoE to simulate the capacity of an ultra-wide MoE while maintaining a lean physical parameter footprint.

2.3 Hybrid Activation and Routing Strategy

The execution of a CS-MoE layer is defined by the simultaneous processing of these two paths. Given a token representation $\mathbf{h}_l \in \mathbb{R}^d$, the routing mechanism manages only the dynamic path (the shared pool).

1. **Shared Path Routing:** The layer-specific router R_l computes scores $\mathbf{s}_l$ for the M shared experts:
$$\mathbf{s}_l = W_{r,l}\mathbf{h}_l$$
where $W_{r,l} \in \mathbb{R}^{M \times d}$. We then apply a Top-$K$ gating function to select the most relevant shared experts:
$$\mathbf{g}_l = \text{Softmax}(\text{TopK}(\mathbf{s}_l, K))$$
2. **Output Aggregation:** The final output of the layer, $\mathbf{o}_l$, is the sum of the dense independent experts and the sparsely activated shared experts:
$$\mathbf{o}_l = \underbrace{\sum_{j=1}^{N_{indep}} \text{Expert}_{e,j}^{(l)}(\mathbf{h}_l)}_{\text{Fixed Path}} + \underbrace{\sum_{i \in \text{TopK}} g_{l,i} \cdot \text{Expert}_{s,i}(\mathbf{h}_l)}_{\text{Dynamic Path}}$$

This design ensures that every token benefits from N_{indep} specialized parameters tailored to the current layer's depth, while simultaneously drawing from K experts in the global pool. By adjusting K and M, the architecture can precisely tune the ratio of compute-to-parameters, effectively breaking the linear relationship between model depth and parameter count.

2.4 Layer-wise Load Balancing for Shared Experts

To prevent the shared pool from suffering from expert collapse - where a few experts dominate the routing across all layers - we implement a layer-wise auxiliary load-balancing loss $\mathcal{L}_{aux}^{(l)}$. This loss is calculated specifically for the shared expert pool at each layer to ensure diverse utilization.

For a batch of T tokens, the balancing loss is defined as:
$$\mathcal{L}_{aux}^{(l)} = M \sum_{i=1}^{M} f_{l,i} \cdot P_{l,i}$$
Where $f_{l,i}$ is the fraction of tokens in the batch that selected shared expert i at layer l, and $P_{l,i}$ is the average routing probability assigned to that expert. The total objective function is:
$$\mathcal{L}_{total} = \mathcal{L}_{LM} + \alpha \sum_{l=1}^{L} \mathcal{L}_{aux}^{(l)}$$
where α is the balancing coefficient. This ensures that the shared pool remains a robust, multi-functional resource throughout the training process.

Table 1: Detailed model hyperparameters (Qwen3-MoE Backbone). *: Top-K activated experts include independent experts of each layer.

Model Type	Total Params	Activated Params	Layers (L)	d_{ffn}	d_{exp}	N_{indep}	Shared Pool (M)	Top-K^*
Dense	0.6B / 1.7B / 4B / 8B	Same	16 - 36	3,072 - 12,288	-	-	-	-
CS-MoE 0.6B	0.6B	0.6B / 0.9B / 1.7B	28	3,072	768	1	84	4 / 10 / 21
CS-MoE 1.7B	1.7B	1.7B / 4B	28	6,144	1,536	1	84	4 / 13
CS-MoE 4B	4B	4B / 8B	36	9,728	2,432	1	108	4 / 10
CS-MoE 8B	8B	4B	36	9,728	2,432	1	324	4
CS-MoE 8B	8B	8B	36	12,288	3,072	1	324	4
CS-MoE 12B	12B	4B	36	9,728	2,432	1	540	4

3 Implementation

3.1 Model Configurations and Backbone

To evaluate the efficacy of **CS-MoE**, we implement three architectural paradigms: **Dense Transformers**, **Standard MoE Transformers**, and our proposed **CS-MoE**. All models are developed using the **Qwen3-MoE**[3] architecture as the foundational backbone, inheriting its optimized implementations of Grouped-Query Attention (GQA), SwiGLU activation functions, and Rotary Positional Embeddings (RoPE).

The configurations across various scales are detailed in Table 1, where d_{ffn} and d_{ffn} are the dimensions of the feed-forward layer and expert, respectively. To reconstruct the standard Qwen3-MoE architecture as our CS-MoE, we split a feed-forward layer into four experts at each layer. For example, CS-MoE 0.6B has four experts per layer with a dimension of 768. Additionally, we employ a **Total (Activated)** notation to delineate the relationship between the physical parameter footprint in high-bandwidth memory (HBM) and the actual computation performed per token.

In the CS-MoE variants, the **Fixed Path** consists of a single independent expert per layer ($N_{indep} = 1$). In contrast, the **Dynamic Path** utilizes a shared expert pool (M) where only K experts are activated per token. This structure allows the 8B CS-MoE variant, with all parameters activated, to achieve competitive performance while activating only approximately **55%** of its parameters.

3.2 Training Protocol

All models are pre-trained on a massive integrated corpus comprising **WuDao** [21] and **DCLM** [1], ensuring a robust mixture of multi-lingual knowledge and high-quality web-scale data. To maintain a rigorous and controlled comparison, we unify the training hyperparameters across all experimental groups, as specified in Table 2.

Table 2: Unified Training Hyperparameters

Hyperparameters	**Settings**
Optimizer	AdamW ($\beta_1 = 0.9, \beta_2 = 0.95, \epsilon =$ 1e-8)
Learning Rate	3e-4
Weight Decay	0.01
Batch Size	512 (effective token count $\approx$ 1M)
Warmup Steps	1,000
Total Steps	12,500
LR Schedule	Cosine Annealing to $0.1 \times \text{LR}_{\max}$
Context Length	2048

The implementation utilizes a customized version of the Megatron library[4] to efficiently handle the routing logic of the shared expert pool across 8 Nvidia H200 GPUs. For MoE-based models, the auxiliary load-balancing coefficient α is fixed at 0.01 to ensure expert utilization without destabilizing the language modeling objective.

[3] `https://github.com/huggingface/transformers/tree/main/src/transformers/models/qwen3_moe`

[4] `https://github.com/NVIDIA/Megatron-LM`

4 Experiments

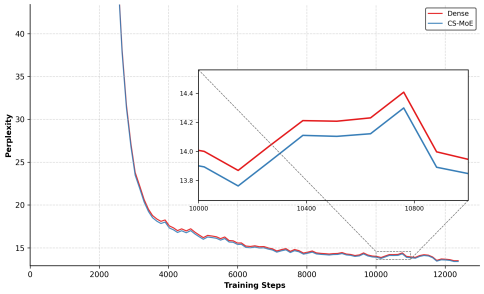
We conduct three distinct groups of experiments to empirically validate the architectural superiority of CS-MoE, focusing on efficiency, scalability, and structural convergence.

To quantify the architectural efficiency of the super-sharing mechanism, we define the **Expert Utilization Ratio (EUR)** (ρ), which represents the proportion of unique shared experts activated across the entire network relative to the total expert pool:

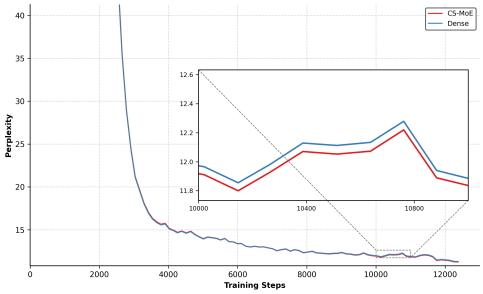
$$\rho = \frac{\left|\bigcup_{l=1}^{L} \mathcal{S}_l\right|}{\min(M, \delta)}$$

where L is the number of layers, N is the number of independent experts per layer, M is the total size of the shared expert pool, and $\mathcal{S}_l$ denotes the subset of kN shared experts activated at layer l. Notably, δ accumulates the activated experts across all layers, which may exceed M as k increases.

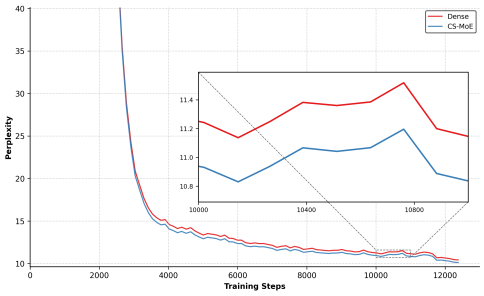
4.1 Efficiency Gains and Parameter Expansion



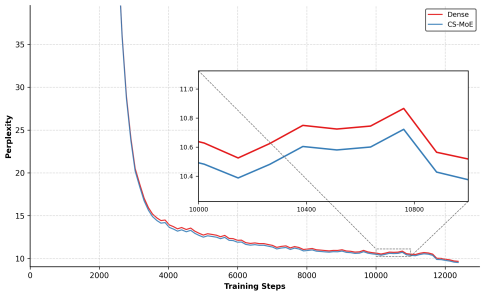
(a) PPL of 0.6B Models.



(b) PPL of 1.7B Models.



(c) PPL of 4B Models.



(d) PPL of 8B Models.

Figure 3: Comparison between CS-MoE and Dense models.

In our first study, we compare CS-MoE against Dense Transformers while keeping the total parameter count and FLOPs aligned. As shown in Figure 3, CS-MoE consistently outperforms Dense baselines across all scales. Firstly, we observe a "Parameter Expansion" effect: for example, the CS-MoE model with an 8B total parameter footprint, but only **55%** of those parameters activated per token, achieves a significantly lower perplexity (PPL) than a Dense model of the same total size. This suggests that the hybrid expert pool allows the model to leverage a wider knowledge base than its dense counterpart without increasing the computational burden per forward pass. Additionally, scaling the size of CS-MoE introduces significantly more performance gain than increasing the activations, suggesting that the breadth of the shared expert

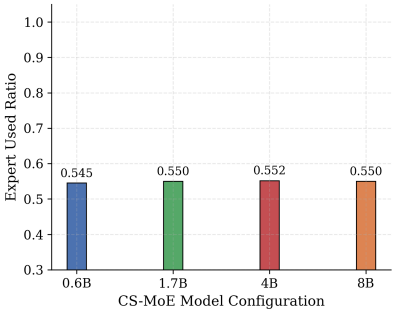
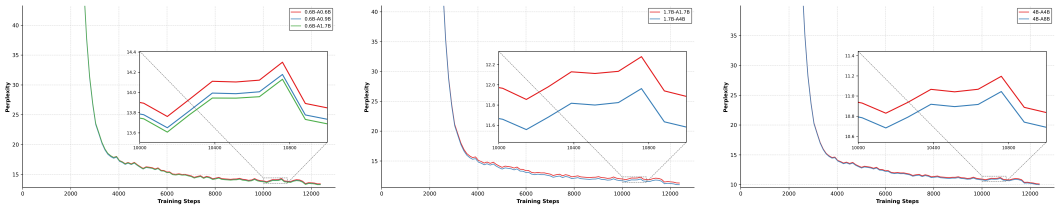


Figure 4: The EUR of CS-MoE increases with scaling model size.

pool is more critical for performance than the density of per-token routing. This highlights the importance of global parameter availability in enhancing model capacity.

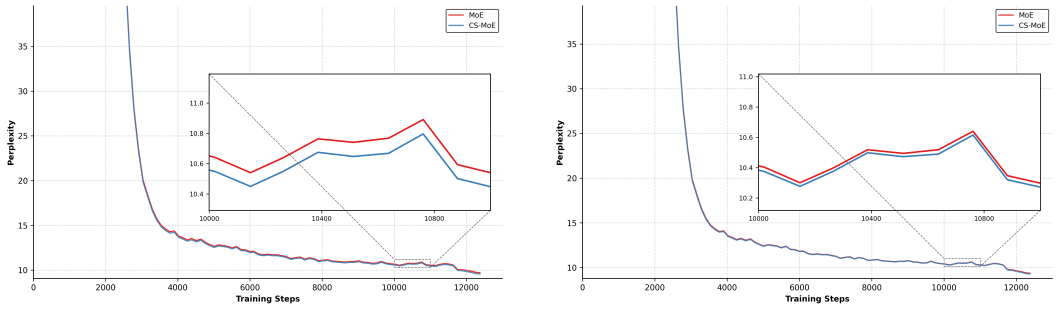


(a) PPL of 0.6B-AxB Models. (b) PPL of 1.7B-AxB Models. (c) PPL of 4B-AxB Models.

Figure 5: Comparison among CS-MoE models with varying activations.

The second group of experiments investigates the relationship between the expert activation count (K) and model performance while holding the total parameter scale fixed. In Figure 5, we increase K in the 0.6B, 1.7B, and 4B CS-MoE variants so that the model can activate more than its actual parameters per token. We then observe a monotonic improvement in performance. Unlike standard architectures where capacity is strictly tied to parameter count, CS-MoE demonstrates that **increasing token-level compute allocation (FLOPs)** - by activating more experts from the shared pool - can effectively bypass the traditional "Parameter-Compute bottleneck." This result confirms that the shared expert pool acts as a scalable resource that the model can query more intensively to yield continuous performance gains.

4.2 Architectural Convergence and Sharing Dynamics



(a) PPL of 8B-A4B Models. (b) PPL of 12B-A4B Models.

Figure 6: Comparison between CS-MoE and Sparse MoE models with equal activations.

Finally, we analyze the convergence behavior of CS-MoE relative to Standard MoEs under fixed FLOP constraints. We increase the model size (as well as the size of the shared expert pool (M)) and measure the repetition rate of expert activation. Results in Figure 7 indicate that as M expands, the **expert utilization ratio** across layers significantly increases to around 1.0. According to Figure 4, the performance of CS-MoE asymptotically approaches that of a standard, layer-isolated MoE. This suggests that CS-MoE functions as a generalized architecture: at small M, it provides extreme parameter efficiency through high reuse; as $M \to \infty$, it preserves the modeling power of a standard MoE. This flexibility allows CS-MoE to define a Pareto frontier that is particularly advantageous in FLOP-constrained or memory-constrained deployment scenarios.

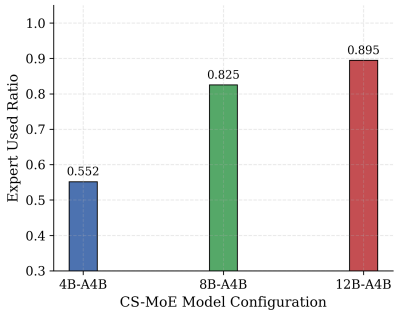


Figure 7: Expert used ratio of CS-MoE models with 4B activations.

5 Related Work

5.1 Mixture-of-Experts Transformers

Mixture-of-Experts (MoE) has emerged as the leading paradigm for scaling Transformer capacity while maintaining constant computational costs per token. Foundational works established the sparsely-gated MoE layer [17] and demonstrated its scalability to hundreds of billions of parameters through automatic sharding [11]. The Switch Transformer [6] further optimized this via Top-1 routing, significantly improving training efficiency over dense baselines. Recent innovations have focused on refining routing granularity and expert specialization. DeepSeekMoE [4] and DeepSeek-V2 [16] introduced fine-grained expert segmentation and shared experts to better capture common knowledge, while Mixtral [8] validated the efficacy of Top-2 routing for open-weight models. Further architectural refinements include OLMoE [13], which provides open-source scaling insights, and MoE++ [9], which utilizes zero-computation experts to reduce FLOPs. Efficiency is further pushed by X-MoE [20] through threshold-based routing and Seqtopk [19], which shifts from token-level to sequence-level routing to minimize communication overhead. Most recently, MiMo-V2-Flash [18] has optimized ultra-sparse designs specifically for agentic workflows. Despite these diversities in routing and granularity, existing MoE architectures share a fundamental constraint: layer-wise isolation. Experts are strictly confined to their respective depths, preventing the exploitation of cross-layer semantic redundancies. Our proposed CS-MoE addresses this limitation by enabling longitudinal expert reusability, thereby maximizing parameter utility across the entire network hierarchy.

5.2 Parameter Sharing in Transformers

Parameter sharing has long served as a vital mechanism for enhancing model generalization and efficiency, with early research focusing on weight tying between embedding and output layers [15] in Transformers. This evolved into the full cross-layer parameter sharing introduced by ALBERT [10], though its reliance on uniform sharing at every depth inherently limits the model's ability to learn specialized representations. While subsequent investigations in DeBERTa [7] and PaLM [2] identified functional similarities across layers, and Universal Transformers [5] proposed recurrent sharing, these approaches often introduce sequential bottlenecks that compromise parallelization. Within the Mixture-of-Experts (MoE) domain, most architectures maintain strict layer-wise isolation of experts [17, 11, 6]. DeepSeekMoE [4] attempted to bridge this gap with "shared experts," yet these remain restricted to intra-layer application and do not exploit the longitudinal redundancies observed in recent analyses like MoE++ [9]. CS-MoE addresses these fundamental constraints by implementing a dual-tier hierarchy that balances independent, depth-specific experts with a global shared pool. Unlike the rigid uniformity of ALBERT or the static recurrence of Universal Transformers, CS-MoE utilizes per-token dynamic routing to enable genuine inter-layer allocation, thereby achieving superior parameter efficiency while preserving the essential hierarchical capacity of deep Transformer networks.

6 Conclusion

This paper presents **CS-MoE**, a novel Transformer architecture designed to address inter-layer parameter redundancy through a dual-tier expert hierarchy. By decoupling depth-localized independent experts from a centralized shared pool, CS-MoE facilitates cross-layer functional reuse while maintaining hierarchical specialization. Extensive evaluations demonstrate that CS-MoE achieves superior perplexity compared to dense baselines while activating fewer parameters. Furthermore, the architecture establishes a flexible Pareto frontier between computational expenditure and model capacity, allowing performance to scale monotonically with the number of expert activations. In the future, we will explore efficient expert parallel mechanisms for CS-MoE and pull it to resource-limited deployment scenarios. Our main directions are:

- **Generalization**. Scaling CS-MoE to ultra-large regimes, such as 30B and 80B parameter models, to examine the robustness of super-sharing at the extreme frontier.
- **Advanced Sharing Strategy**. The impact of non-uniform allocation by varying the number of independent experts at different layers.

- **Depth-aware routing**, which involves interventions to modulate expert reuse patterns across network stages, further refining the balance between localized specialization and global knowledge sharing.

References

[1] Apple, University of Washington, Tel Aviv University, et al. Datacomp-lm: In search of the next generation of training sets for language models. *arXiv preprint arXiv:2406.11794*, 2024.

[2] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. PaLM: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023.

[3] OpenReview Contributors. Sparseswaps: Tractable llm pruning mask refinement at scale. In *The Thirteenth International Conference on Learning Representations*, 2025.

[4] Damai Dai, Chengqi Deng, Shuo Zhao, et al. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*, 2024.

[5] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019.

[6] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[7] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. DeBERTa: Decoding-enhanced BERT with disentangled attention. In *International Conference on Learning Representations*, 2021.

[8] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mixtral of experts, 2024.

[9] Peng Jin, Bo Zhu, Li Yuan, and Shuicheng Yan. Moe++: Accelerating mixture-of-experts methods with zero-computation experts, 2024.

[10] Zhenzhong Lan, Mingdao Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. ALBERT: A lite BERT for self-supervised learning of language representations. In *International Conference on Learning Representations*, 2020.

[11] Dmitry Lepikhin, Hyoukseok Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.

[12] Wang Ling, Dani Yogatama, Chris Dyer, and Phil Blunsom. Program induction by rationale generation: Learning to solve and explain algebraic word problems. In Regina Barzilay and Min-Yen Kan, editors, *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics, ACL 2017, Vancouver, Canada, July 30 - August 4, Volume 1: Long Papers*, pages 158–167. Association for Computational Linguistics, 2017.

[13] Niklas Muennighoff et al. Olmoe: Open mixture-of-experts language models. *arXiv preprint arXiv:2501.00662*, 2025.

[14] Van Bach Nguyen, Christin Seifert, and Jörg Schlötterer. CEval: A benchmark for evaluating counterfactual text generation. In Saad Mahamood, Nguyen Le Minh, and Daphne Ippolito, editors, *Proceedings of the 17th International Natural Language Generation Conference*, pages 55–69, Tokyo, Japan, September 2024. Association for Computational Linguistics.

[15] Ofir Press and Lior Wolf. Using the output embedding to improve language models. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 157–163. Association for Computational Linguistics, 2017.

[16] Zhihong Shao, Damai Dai, Daya Guo, Bo Liu (Benjamin Liu), Zihan Wang, and Huajian Xin. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *ArXiv*, abs/2405.04434, 2024.

[17] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.

[18] Xiaomi AI Team. Mimo-v2-flash: Ultra-sparse moe for agentic workflows. *Technical Report*, 2026.

[19] Tiansheng Wen, Yifei Wang, Aosong Feng, Long Ma, Xinyang Liu, Yifan Wang, Lixuan Guo, Bo Chen, Stefanie Jegelka, and Chenyu You. Route experts by sequence, not by token, 2025.

[20] Yuanhang Yang, Shiyi Qi, Wenchao Gu, Chaozheng Wang, Cuiyun Gao, and Zenglin Xu. XMoE: Sparse models with fine-grained and adaptive expert selection. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Findings of the Association for Computational Linguistics: ACL 2024*, pages 11664–11674, Bangkok, Thailand, August 2024. Association for Computational Linguistics.

[21] Sha Yuan, Hanyu Zhao, Zhengxiao Du, Ning Ding, Xiao Liu, Yukuo Cen, Xu Huang, et al. Wudaocorpora: A super large-scale chinese corpora for pre-training language models. *AI Open*, 2:65–68, 2021.

[22] Longfei Yun, Yonghao Zhuang, Yao Fu, Eric P. Xing, and Hao Zhang. Toward inference-optimal mixture-of-expert large language models. *ArXiv*, abs/2404.02852, 2024.

A Detailed Implementation of the Pilot Study

We conducted our analysis using a pre-trained Qwen3-MoE-4B model, which features 36 Transformer layers and 4 experts per layer ($N = 4$, Top-$K = 1$). The study was performed on a held-out validation set of 50,000 tokens sampled from the DCLM corpus to ensure statistical significance in activation patterns.

Representational Similarity Analysis. To quantify the functional overlap between experts without being misled by the permutation invariance of neural weights, we employed Centered Kernel Alignment (CKA). Unlike Cosine similarity, CKA is invariant to orthogonal transformations and isotropic scaling, making it the standard for comparing internal representations.

For two expert weight matrices $W_i^{(l)}$ and $W_j^{(l')}$ from layers l and l', we computed the linear CKA score as follows:

$$CKA(K, L) = \frac{HSIC(K, L)}{\sqrt{HSIC(K, K)HSIC(L, L)}}$$

where $K = XX^T$ and $L = YY^T$ are the Gram matrices of the activations produced by the respective experts, and $HSIC$ is the Hilbert-Schmidt Independence Criterion. A high CKA score (> 0.85) between a shallow-layer expert and a deep-layer expert served as our primary indicator of longitudinal redundancy.

Zero-Shot Expert Substitution Protocol. To verify the interchangeability of experts, we implemented a greedy substitution algorithm:

1. **Selection:** For each expert in the target "Deep" layers (layers 24-32), we identified a "Source" expert from the "Shallow" layers (layers 1-8) that maximized the CKA score.

Table 3: Downstream benchmark performance (accuracy, %) of CS-MoE and Dense Transformers across pre-training steps. CS-MoE variants use the "-AxYB" notation, where Y denotes the activated parameter count.

Steps	**0.6B / -A0.9B**		**1.7B / -A4B**		**4B / -A8B**	
	Dense	CS-MoE	Dense	CS-MoE	Dense	CS-MoE
CVEAL						
2,500	28.2	31.5	30.1	31.7	29.3	29.8
5,000	32.6	36.8	33.9	34.2	33.1	34.3
7,500	35.1	39.3	36.4	40.1	35.6	41.0
10,000	36.8	41.0	37.7	41.5	38.8	42.4
12,500	37.5	41.9	38.6	42.3	40.7	43.2
AQuA						
2,500	22.4	26.1	21.8	28.5	27.2	33.6
5,000	26.7	31.3	26.5	31.1	32.9	33.8
7,500	29.1	34.0	29.8	34.3	36.1	37.2
10,000	30.5	35.6	31.7	36.5	38.3	39.6
12,500	31.2	36.4	32.6	37.8	39.1	40.5

2. **Replacement:** We performed a direct parameter swap, replacing the weights of the Deep expert with the weights of its Shallow counterpart.
3. **Inference:** We measured the change in model perplexity on the validation set without any gradient updates or fine-tuning.
4. **Baseline:** As a control, we performed random substitutions of equal magnitude.

The moderate increase in perplexity (under 15 within the acceptable region) during targeted substitution, compared to a sharp divergence in the random baseline, empirically justifies the feasibility of the shared expert pool proposed in CS-MoE.

B More Evaluation Results

In addition to the perplexity-based evaluation presented in the main text, we further assess CS-MoE on two widely used foundational benchmarks to verify that the architectural advantages observed during pre-training generalize to downstream task performance.

- **CVEAL** [14] is a comprehensive Chinese evaluation suite that assesses models across 28 categories spanning safety, honesty, helpfulness, and toxicity. It comprises both multiple-choice and open-ended generation tasks, providing a holistic view of a model's ability to align with human values and generate contextually appropriate Chinese text. We report the overall accuracy across all categories.
- **AQuA** [12] is an algebraic reasoning benchmark consisting of 100,000 natural language math problems with multiple-choice answers. Each problem requires multi-step logical reasoning and mathematical computation, serving as a rigorous test of models' quantitative reasoning capabilities. We report the test-set accuracy using 5-shot prompting.

Table 3 reports the evaluation results at five training checkpoints spanning the entire pre-training process. First, CS-MoE surpasses the Dense baseline at every checkpoint for all three model scales on both benchmarks. Notably, at the final checkpoint on CVEAL, the performance gap is most pronounced at the 0.6B scale (+4.4 points: 41.9% vs. 37.5%), and remains substantial at 1.7B (+3.7 points) and 4B (+2.5 points). On AQuA, the gap ranges from +1.4 points (4B) to +5.2 points (0.6B and 1.7B) at the final checkpoint. Second, the performance margin is most pronounced at the 0.6B scale on both benchmarks. CS-MoE 0.6B-A0.9B achieves 41.9% on CVEAL versus 37.5% for Dense 0.6B, and 36.4% on AQuA versus 31.2%. This suggests that the shared expert pool provides a proportionally larger representational benefit when the underlying parameter budget is constrained, effectively compensating for the limited capacity of smaller models. Third, CS-MoE models reach Dense's final-stage performance at substantially earlier training checkpoints. For example, CS-MoE

0.6B-A0.9B achieves 36.8% on CVEAL after only 5,000 steps, already approaching Dense 0.6B's final performance of 37.5% at 12,500 steps. Similarly, on AQuA, CS-MoE 1.7B-A4B attains 31.1% after 5,000 steps, matching Dense 1.7B's final accuracy of 32.6%. This early-stage convergence indicates that the cross-layer sharing of diverse expert representations accelerates knowledge acquisition during pre-training. Finally, CS-MoE's downstream performance improves steadily across training steps at all scales, and the gap with Dense baselines generally widens as training progresses. For instance, on CVEAL at the 1.7B scale, the advantage grows from +1.6 points at 2,500 steps to +3.7 points at 12,500 steps, confirming that the benefits of the shared expert pool compound over longer training horizons.

These results corroborate the findings from the main experiments: CS-MoE's dual-tier expert hierarchy not only reduces pre-training perplexity more efficiently but also translates these gains into superior downstream task performance across diverse evaluation dimensions.